CS 170 - HW 2

Jie Wen

Sep 13, 2025

1 Study Group

None

2 Squaring vs. multiplying matrices

2.1 (a) Five multiplications for a 2×2 square.

Write

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \qquad A^2 = \begin{pmatrix} a^2 + bc & ab + bd \\ ca + dc & cb + d^2 \end{pmatrix} = \begin{pmatrix} a^2 + bc & b(a + d) \\ c(a + d) & cb + d^2 \end{pmatrix}.$$

The two off-diagonal entries both contain the factor $a + d$, the two diagonal entries each contain the common term bc.

So compute the following five scalar products (five multiplications):

$$\begin{aligned} p_1 &= b \cdot (a + d), \\ p_2 &= c \cdot (a + d), \\ p_3 &= b \cdot c, \\ p_4 &= a \cdot a = a^2, \\ p_5 &= d \cdot d = d^2. \end{aligned}$$

Then

$$A^2 = \begin{pmatrix} p_4 + p_3 & p_1 \\ p_2 & p_5 + p_3 \end{pmatrix},$$

and the remaining operations are additions/subtractions only. Thus 5 multiplications suffice.

2.2 (b) What is wrong with the "5-subproblem" divide-and-conquer claim?

The proposed algorithm assumes that matrix squaring can be recursively reduced to exactly five squarings of $\frac{n}{2} \times \frac{n}{2}$ submatrices, analogous to how Strassen's

algorithm reduces matrix multiplication to seven multiplications of $\frac{n}{2} \times \frac{n}{2}$ submatrices. However, this is incorrect because partitioning an $n \times n$ matrix A into four $\frac{n}{2} \times \frac{n}{2}$ blocks P, Q, R, S (so $A = \begin{pmatrix} P & Q \\ R & S \end{pmatrix}$) yields $A^2 = \begin{pmatrix} P^2 + QR & PQ + QS \\ RP + SR & RQ + S^2 \end{pmatrix}$. Computing this requires squarings (P^2, S^2) and general matrix multiplications of distinct matrices (QR, PQ, QS, RP, SR, RQ), not solely squarings. There is no known divide-and-conquer method that expresses the square solely in terms of five recursive squarings of half-sized matrices, so the recurrence $T(n) = 5T(\frac{n}{2}) + O(n^2)$ (leading to $O(n^{\log_2 5})$) does not apply.

2.3 (c) Squaring is as hard as multiplying (reduction).

Let X and Y be arbitrary $n \times n$ matrices. Form the $2n \times 2n$ block matrix

$$B = \begin{pmatrix} 0 & X \\ Y & 0 \end{pmatrix},$$

where 0 denotes the $n \times n$ zero matrix. Compute B^2. Block multiplication gives

$$B^2 = \begin{pmatrix} XY & 0 \\ 0 & YX \end{pmatrix}.$$

Thus the upper-left $n \times n$ block of B^2 is exactly the product XY we wanted.

If squaring a $2n \times 2n$ matrix takes $O((2n)^c) = O(n^c)$ time (constant factor 2^c absorbed in big-O), then now can obtain XY in $O(n^c)$ time by forming B and computing B^2. Hence a squaring algorithm with exponent c yields a multiplication algorithm with the same exponent c. Therefore squaring is no easier (in asymptotic exponent) than general multiplication.

3 Satellite diagnostics

3.1 (a) Single Faulty Sensor

- Partition the n sensors into two halves.
- Test one half:
  - If the test **fails**, the faulty sensor is in that half.
  - If the test **passes**, the faulty sensor is in the other half.
- Repeat this process recursively on the half containing the faulty sensor.

Each test halves the number of candidates, so the number of tests is $O(\log n)$.

3.2 (b) Guaranteed Passing Tests

Since there are s faulty sensors, the worst-case scenario (minimizing passing tests) occurs when faulty sensors are spread across as many groups as possible:

- If $s < x$, at most s groups contain faulty sensors $\Rightarrow$ at least $x - s$ groups are fault-free.
- If $s \geq x$, all groups may contain at least one faulty sensor $\Rightarrow$ no groups are guaranteed fault-free.

Thus, the number of tests that are guaranteed to pass is: $\max(0, x - s)$

3.3 (c) Identify all s faulty sensors in $O\big(s \log(n/s)\big)$ tests.

3.3.1 (i) Strategy (assume s is known).

Repeat the following until the remaining unknown set S of sensors has $|S| \leq 2s$:

1. Partition the current set S into $x = 2s$ disjoint groups of (as close as possible to) equal size $|S|/x$.
2. Test every group in parallel (that is, perform x tests, one per group).
3. Any group whose test **passes** contains zero faulty sensors — remove all sensors in those passing groups from further consideration. Keep the sensors in failing groups for the next iteration.

When $|S| \leq 2s$ finish by testing each sensor in S individually (at most $2s$ further tests) to find all remaining faulty sensors.

3.3.2 (ii) Proof that the strategy works.

By part (b) with $x = 2s$, the number of passing groups is at least $x - s = s$ (provided $s \leq x$, which holds since $x = 2s$). Each passing group has $\approx |S|/x$ sensors, so the total number of sensors removed that round is at least

$$s \cdot \frac{|S|}{x} = s \cdot \frac{|S|}{2s} = \frac{|S|}{2}.$$

Thus each round removes at least half of the remaining sensors. Therefore after t rounds the remaining number of sensors is at most $|S|/2^t$. Repeating until $|S| \leq 2s$ therefore requires $O(\log(n/s))$ rounds. Once $|S| \leq 2s$, testing each individually reveals which are faulty (since with $\leq 2s$ sensors and at most s faults, individual tests suffice).

3.3.3 (iii) Worst-case number of tests (analysis).

Each round uses $x = 2s$ group-tests. Number of rounds until $|S| \le 2s$ is at most

$$\left\lceil \log_2 \frac{n}{2s} \right\rceil \le \lceil \log_2(n/s) \rceil.$$

So tests used in the halving phase $\le 2s \cdot \lceil \log_2(n/s) \rceil$. The final phase tests at most $2s$ individual sensors. Hence total tests

$$\le 2s \cdot \lceil \log_2(n/s) \rceil + 2s = O\big(s \log(n/s)\big).$$

This bound is tight up to constant factors and matches the stated target.

4 Two sorted arrays

4.1 Algorithm Description

Let the two sorted arrays be A and B, each has size k, with distinct integers. My algorithm proceeds recursively by comparing elements at strategic positions to eliminate a portion of one array that cannot contain the k-th smallest element.

- **Base Cases**:
  - If one array is empty, the k-th element is simply the k-th element of the other array.
  - If $k = 1$, return the smaller of the first elements of A and B.
- **Recursive Step**:
  - Ensure A is the shorter array (swap if necessary).
  - Let $i = \min(\text{len}(A), \lfloor k/2 \rfloor)$ and $j = k - i$.
  - Compare $A[i-1]$ and $B[j-1]$:
    - If $A[i-1] < B[j-1]$, discard the first i elements of A and recurse to find the $(k-i)$-th smallest in the remaining arrays.
    - Otherwise, discard the first j elements of B and recurse to find the $(k-j)$-th smallest.

4.2 Proof of Correctness

Use induction on k.

- **Base Cases**: When $k = 1$ or one array is empty, the result is trivially correct.
- **Inductive Step**: Assume the algorithm works for all values less than k. Consider the recursive step where $i = \min(\text{len}(A), \lfloor k/2 \rfloor)$ and $j = k - i$. Without loss of generality, assume $A[i-1] < B[j-1]$.

- The elements $A[0], \ldots, A[i-1]$ are all $\leq A[i-1]$, and $B[0], \ldots, B[j-1]$ are all $\leq B[j-1]$. Since $A[i-1] < B[j-1]$, all elements in $A[0..i-1]$ are less than $B[j-1]$.
- Thus, the number of elements strictly less than $B[j-1]$ is at least i (from A) plus $j-1$ (from B), totaling $i+j-1 = k-1$.
- Therefore, $B[j-1]$ is at least the k-th smallest element, and any element in $A[0..i-1]$ is among the first $k-1$ smallest elements. Hence, the k-th smallest cannot be in $A[0..i-1]$, and we can safely discard these i elements.

The symmetric argument holds when $A[i-1] > B[j-1]$. By induction, the algorithm is correct.

4.3 Runtime Analysis

In each recursive call, reduce k by at least $\lfloor k/2 \rfloor$. The recurrence is:

$$T(k) = T(k/2) + O(1)$$

This recurrence resolves to $O(\log k)$. Since each step involves constant-time comparisons and index arithmetic, the overall time complexity is $O(\log k)$, which is better than $O(k)$.

4.4 Pseudocode

```
def findKth(A, nA, B, nB, k):
    # Ensure A is the shorter array
    if nA > nB:
        return findKth(B, nB, A, nA, k)

    # Base cases
    if nA == 0:
        return B[k-1]
    if k == 1:
        return min(A[0], B[0])

    # Choose indices to compare
    i = min(nA, k // 2)
    j = k - i

    if A[i-1] < B[j-1]:
        # Discard first i elements of A
        return findKth(A[i:], nA - i, B, nB, k - i)
    else:
        # Discard first j elements of B
        return findKth(A, nA, B[j:], nB - j, k - j)
```

Initial Call:
`findKth(A, k, B, k, k)`

5 The magical keys and locks

5.1 (a)

Consider the set of n keys and n locks. Select the given lock L for which there are r keys too small. Try all n keys in L, using n comparisons. This identifies the matching key M for L (the perfect fit), the r keys smaller than M (too small for L), and the $n - r - 1$ keys larger than M (too big for L).

Next, try M in each of the remaining $n - 1$ locks, using $n - 1$ comparisons. For each such lock L':

- If M is too big for L', then L' requires a key smaller than M (so L' is one of the r smaller locks).
- If M is too small for L', then L' requires a key larger than M (so L' is one of the $n - r - 1$ larger locks).

This partitions the remaining locks into r smaller locks and $n - r - 1$ larger locks. The r smaller keys match the r smaller locks, and the $n - r - 1$ larger keys match the $n - r - 1$ larger locks. Pair M with L, and reduce to the two subproblems on these subsets. Total comparisons: $2n - 1 = O(n)$.

5.2 (b)

The input is an array of $2n$ items (n keys and n locks in random order), but since keys and locks are distinguishable types, separate them into a set K of n keys and a set L of n locks.

The randomized algorithm to match them is recursive (on subsets of m keys and m locks):

- Base cases: If $m = 0$, return empty. If $m = 1$, try the key in the lock (it fits) and return the pair.
- For $m = n \geq 2$: Uniformly at random, pick a lock L_i from the n locks. Try all n keys in L_i (n comparisons), identifying its matching key M and letting r be the number of keys too small for L_i. Then try M in the other $n-1$ locks ($n-1$ comparisons), partitioning the remaining keys and locks into a smaller subproblem of size r and a larger subproblem of size $n - r - 1$, as in (a). Recursively solve the two subproblems and return their pairs plus the pair (M, L_i).

Assuming the input is randomly ordered (equivalent to random permutations of sizes), the rank of the chosen lock's required key size is uniform from 1 to n, so r is uniform from 0 to $n - 1$. Thus, the expected runtime $A(n)$ satisfies

$$A(n) = \frac{1}{n} \sum_{r=0}^{n-1} (A(r) + A(n - r - 1)) + (2n - 1),$$

with $A(0) = 0$ and $A(1) = 1$.

5.3 (c)

The recurrence simplifies to $A(n) = \frac{2}{n}\sum_{k=0}^{n-1} A(k) + (2n-1)$. The assumption implies A is convex (sums $A(a)+A(b)$ with $a+b$ fixed are minimized when $|a-b|$ is minimized and maximized when $|a-b|$ is maximized). For the quicksort-like recurrence, convexity ensures the expected cost over uniform random pivots (splits) is $O(n \log n)$, as the average cost is bounded above by a multiple of the balanced case cost (which solves to $O(n \log n)$) plus adjustments from convexity properties in the induction or closed-form solution.

5.4 (d)

Prove by strong induction that $A(n) \leq 4nH_n$, where $H_n = \sum_{i=1}^{n} \frac{1}{i}$ (and $H_0 = 0$).

Base cases: $A(0) = 0 \leq 0$. $A(1) = 1 \leq 4 \cdot 1 \cdot 1 = 4$.

Assume for all $m < n$, $A(m) \leq 4mH_m$. Then

$$A(n) = \frac{2}{n}\sum_{k=0}^{n-1} A(k) + (2n-1) \leq \frac{2}{n}\sum_{k=1}^{n-1} 4kH_k + (2n-1).$$

Now,

$$\sum_{k=1}^{n-1} kH_k = \frac{n(n-1)}{2}H_{n-1} - \frac{(n-1)(n-2)}{4}.$$

So

$$\sum_{k=1}^{n-1} 4kH_k = 2n(n-1)H_{n-1} - (n-1)(n-2),$$

and

$$A(n) \leq \frac{2}{n}[2n(n-1)H_{n-1} - (n-1)(n-2)] + (2n-1) = 4(n-1)H_{n-1} - 2\frac{(n-1)(n-2)}{n} + 2n - 1.$$

We need to show this is $\leq 4nH_n = 4nH_{n-1} + 4$. The difference is

$$[4(n-1)H_{n-1} - 2\frac{(n-1)(n-2)}{n} + 2n - 1] - [4nH_{n-1} + 4] = -4H_{n-1} - 2\frac{(n-1)(n-2)}{n} + 2n - 5.$$

Substituting $\frac{(n-1)(n-2)}{n} = n - 3 + \frac{2}{n}$ gives

$$-4H_{n-1} - 2(n - 3 + \frac{2}{n}) + 2n - 5 = -4H_{n-1} + 1 - \frac{4}{n} \leq 0$$

for $n \geq 2$ (since $H_{n-1} \geq 1$). Thus, $A(n) \leq 4nH_n = O(n \log n)$.